

# Week 9

## Support Vector Machines

*Finding the Widest Street*

---

| Section                                  | Question it answers                                           |
|------------------------------------------|---------------------------------------------------------------|
| 1. From Parameters to Margins            | Which boundary should we trust when signals drift?            |
| 2. Testing SVM on Our Data               | Does maximum-margin classification work on our reaching data? |
| 3. The Kernel Trick                      | How can a linear method capture nonlinear boundaries?         |
| 4. Dimensionality Revisited              | Why do SVMs survive high dimensions better than KNN?          |
| 5. Confusion Matrices                    | Which reach directions are systematically confusable?         |
| 6. From Offline Decoding to BCI Pipeline | When does margin translate to real-time robustness?           |
| 7. Bringing It Together                  | Where does SVM fit in the comparison table?                   |

# 1. From Parameters to Margins

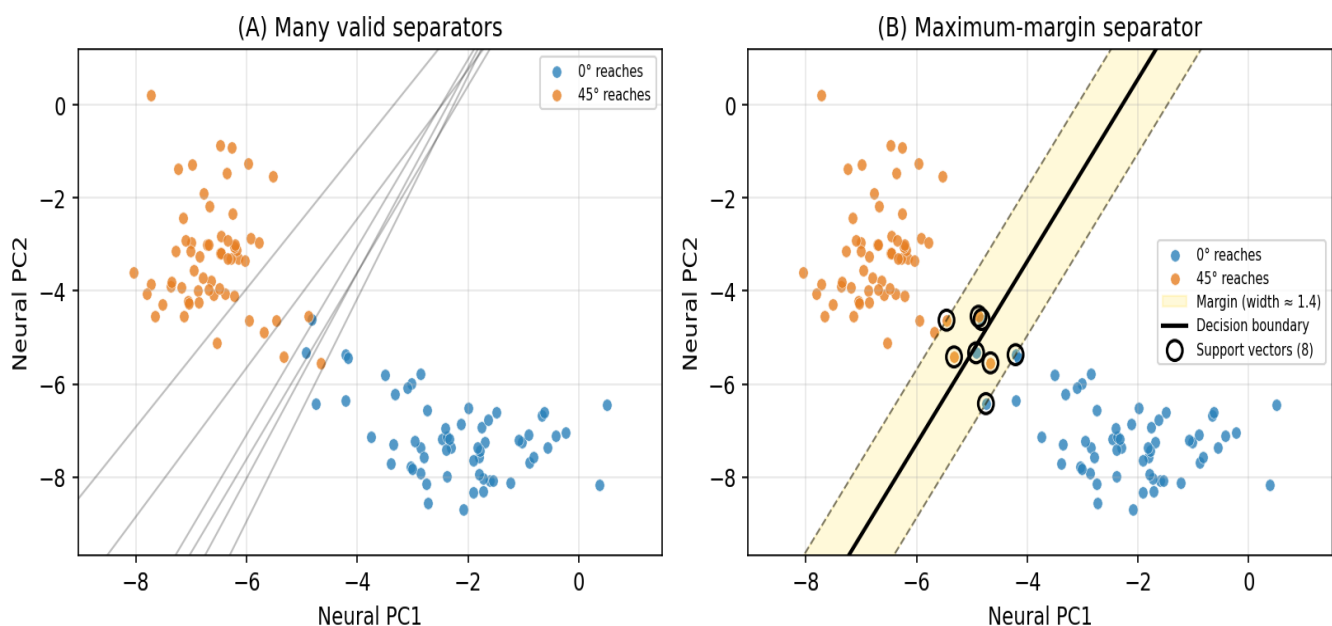
## Which boundary should we trust when signals drift?

Every classifier we have built so far makes a commitment about where to draw the line between movement classes. Logistic regression learns a weight vector — seven numbers for EMG binary classification (six weights plus an intercept). Naive Bayes learns means and variances — twelve numbers for the same task. KNN stores the entire training set and defers the commitment to prediction time. Each approach produced a boundary that correctly separated most of our training trials. But here is a question none of them answered: when multiple boundaries can classify the training data equally well, which one should we actually use?

This is not an abstract concern. In a motor control experiment, the EMG signals you record on Monday morning are not identical to the signals you record on Monday afternoon. Electrodes shift. Skin impedance changes with sweat. Muscles fatigue and alter their recruitment patterns. The neural population vector for a 90° reach drifts as the subject's attention wanes or as cortical excitability fluctuates across the session. Every one of these effects pushes the feature vector for a given movement slightly away from where it was during calibration. If your classifier's boundary happens to pass close to some of these trials, even a small drift can flip the prediction.

Support vector machines address this directly. Instead of finding any boundary that separates the classes, the SVM finds the boundary that sits as far as possible from the nearest trials of both classes. This gap is called the margin, and the SVM maximises it. The intuition maps onto a concrete motor control scenario: imagine drawing a street between the healthy and impaired clusters in EMG space. A narrow alley means that the slightest electrode shift sends a trial to the wrong side. A wide boulevard gives you room — the signal can drift and the classification still holds.

Adjacent Reach Directions (0° vs 45°): Why the Widest Margin Matters



*Figure 1. Two adjacent reach directions (0° vs 45°) in neural PCA space. (A) Many valid separating hyperplanes exist — previous methods give us one, but which? (B) The SVM selects the one with the widest margin. Only 8 trials (circled) are support vectors; every other trial could be removed without changing the boundary.*

Consider what happened in our data across prior weeks. Logistic regression achieved 85% on EMG binary diagnosis and 91% on neural 8-direction decoding. KNN hit 100% on EMG binary because healthy and impaired subjects occupy well-separated clusters. But none of these methods asked how much room there is between the boundary and the nearest misclassifiable trial. Logistic regression, for example, finds the maximum-likelihood separator — but that boundary could pass arbitrarily close to the nearest trial as long as the probability assignment is correct.

## The optimisation problem

Formally, given training data  $\{(x_i, y_i)\}$  with  $y \in \{-1, +1\}$ , a linear SVM solves: minimise  $\frac{1}{2} \|w\|^2$  subject to  $y_i(w \cdot x_i + b) \geq 1$  for all  $i$ . The margin width is  $2/\|w\|$ . Minimising  $\|w\|^2$  therefore maximises the margin. The constraints ensure every trial is on the correct side.

This is a fundamentally different objective from anything we have seen. Logistic regression maximises the probability of the observed labels. Naive Bayes models each class via independent Gaussian likelihoods and applies Bayes' theorem. The SVM maximises geometric distance to the boundary — a quantity that directly relates to how much your signals can drift before decoding fails.

In BCI research, this connection between margin and robustness was recognised early. Electrode impedance can drift by 10–20% within a single recording session. A classifier with a wider margin tolerates larger perturbations in feature space before a decoding error occurs. This is one reason SVMs became the dominant classifier in BCI competitions between 2003 and 2015, especially for motor imagery paradigms where signals are inherently noisy and non-stationary.

## Support vectors: the trials that matter

The SVM optimisation has constraints — one per training trial — requiring that each trial is classified correctly with at least unit margin. To solve a constrained optimisation like this, we use Lagrange multipliers: we assign a multiplier  $\alpha_i \geq 0$  to each constraint and fold it into the objective. Intuitively,  $\alpha_i$  measures how much constraint  $i$  matters. If a trial sits comfortably inside its class region (constraint not tight), the solver sets  $\alpha_i = 0$ . If the trial sits exactly on the margin boundary (constraint tight),  $\alpha_i > 0$ . The result is the dual formulation of the SVM, which expresses the solution entirely in terms of these multipliers.

The remarkable consequence: the boundary depends on only the trials with  $\alpha_i > 0$  — those sitting exactly on the margin edge. These are the support vectors. Every other trial is irrelevant; you could delete it without changing the boundary at all.

We saw this in Figure 1: only 8 support vectors out of 120 trials for the  $0^\circ$  vs  $45^\circ$  binary problem. For the full 8-direction task across all 480 trials, the linear SVM uses 206 support vectors — about 43% of the data. The fraction is higher because eight overlapping direction clusters produce more ambiguous trials at class boundaries. But even here, over half the training set is irrelevant. The cleanly stereotyped trials — a textbook  $90^\circ$  reach with a strong, consistent population vector — sit safely inside the margin and do not influence the classifier at all.

This sparsity has a practical consequence: the dual formulation depends on data only through dot products  $x_i \cdot x_j$ . We will exploit this in Section 3, where the kernel trick lets us replace these dot products with kernel functions that implicitly operate in higher-dimensional spaces — giving us nonlinear boundaries without the computational cost of explicitly computing the new features.

## Where prior classifiers drew their boundaries

Before we see SVM results, let us revisit how our previous classifiers partition the PCA-projected neural space for the 8-direction decoding task. Logistic Regression carves a straight boundary; Naive Bayes produces curved (quadratic) boundaries; KNN produces the irregular, locally-adaptive regions we studied last week. None of these methods explicitly maximise the gap between direction clusters. The SVM will.

## Prior Classifiers: 8-Direction Decoding in Neural PCA Space

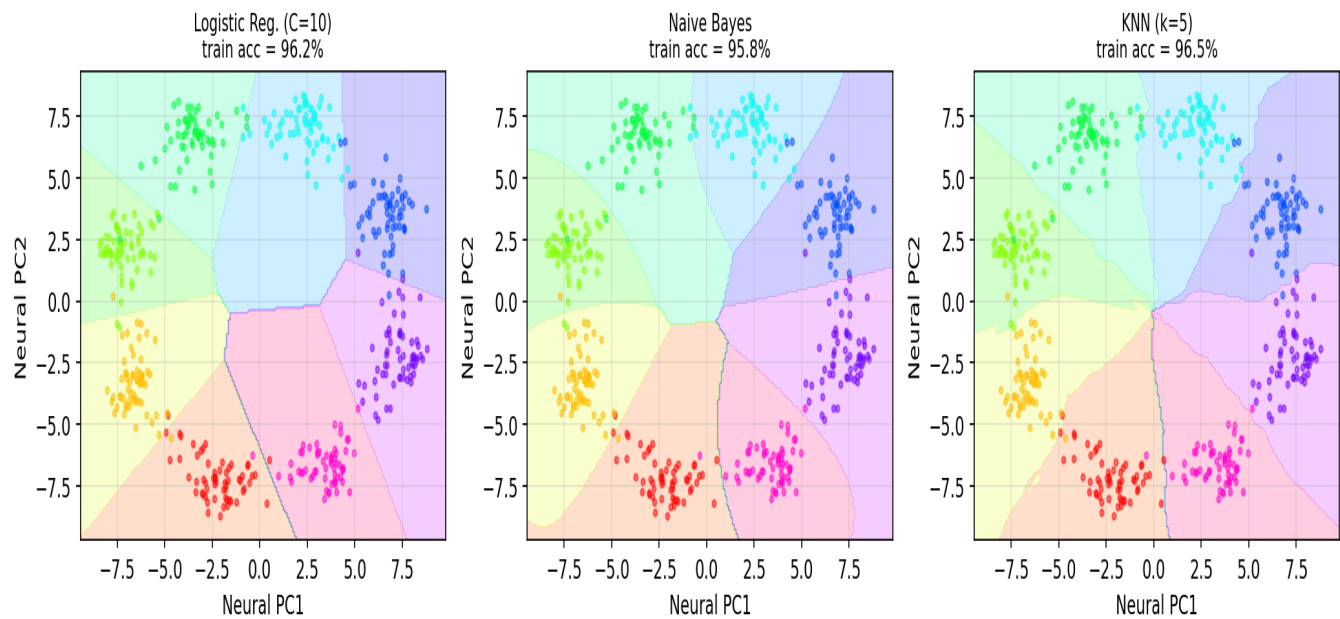


Figure 2. Decision boundaries for three prior classifiers in neural PCA space (80 neurons projected to 2 PCs). LR produces straight boundaries; Naive Bayes produces curved (quadratic) boundaries; k-NN produces jagged, locally-fit regions. None of these methods explicitly maximise the margin between direction clusters.

## 2. Testing SVM on Our Data

In Section 1 we saw that the SVM finds the maximum-margin boundary between two adjacent reach directions using 80 neural firing rates projected to 2 PCs. Now we apply the SVM to our full dataset: all 8 directions, all 20 subjects, both EMG and neural features, and both tasks (direction decoding and clinical diagnosis). The question is whether the margin principle — choosing the boundary with the most room for electrode drift and neural variability — actually translates to better LOSO accuracy than the methods we already have.

Before we run the comparison, we need to choose a value for  $C$ . This is the SVM's one tunable knob, and it controls the balance between margin width and training accuracy. In ML terminology,  $C$  is called a hyperparameter rather than a parameter. The distinction matters: parameters are the numbers the algorithm learns from data — the weight vector  $w$  and intercept  $b$  that define the boundary. Hyperparameters are choices we make before training that control how the algorithm behaves — like  $k$  in KNN (Week 8) or the regularisation strength  $C$  in logistic regression (Week 5). The algorithm cannot learn these from the training data; we select them via cross-validation.

A large  $C$  penalises misclassifications heavily, forcing a narrow margin that classifies every training trial correctly (hard margin). A small  $C$  allows more violations, trading training accuracy for a wider margin and better generalisation to held-out subjects. Think of it as a dial between “trust every trial in the calibration session” and “expect some noise and build in slack.”

In the soft-margin formulation the objective becomes: minimise  $\frac{1}{2} ||w||^2 + C \sum \xi_i$ , subject to  $y_i(w \cdot x_i + b) \geq 1 - \xi_i$ ,  $\xi_i \geq 0$ . The slack variables  $\xi_i$  allow individual points to violate the margin. At  $C \rightarrow \infty$  we recover the hard-margin SVM; at  $C \rightarrow 0$  we essentially ignore the data.

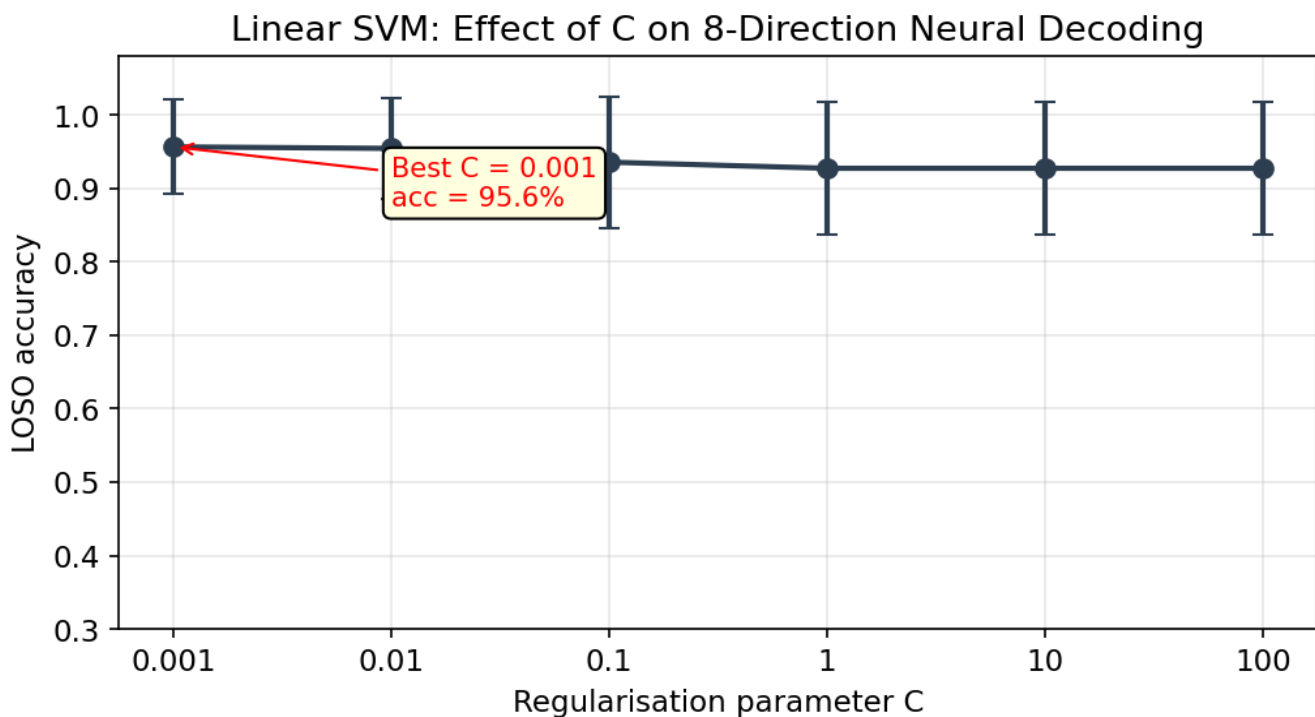


Figure 3. LOSO accuracy as a function of  $C$  for linear SVM on neural 8-direction decoding. Very small  $C$  underfits (too much slack); very large  $C$  overfits to training subjects. The sweet spot balances margin width against training error.

C also controls how many trials become support vectors. Remember from Section 1 that the linear SVM at  $C = 1$  used 206 of 480 trials (43%) as support vectors for the full 8-direction task. That number is not fixed — it changes with  $C$ . At  $C = 0.001$  (maximum slack), every trial is a support vector: the margin is so wide that all 480 points sit inside it. At  $C = 0.01$ , the count drops to 287 (60%). By  $C = 0.1$  it stabilises at around 206 and stays there as  $C$  increases further. This makes intuitive sense: once  $C$  is large enough to correctly classify every training trial, adding more penalty cannot remove any support vectors — the 206 ambiguous trials are genuinely close to a class boundary and will remain support vectors no matter how hard you push.

## Method comparison

How does the linear SVM stack up against Logistic Regression (Week 5), Naive Bayes (Week 7), and KNN (Week 8)? We use  $C = 1$  as a reasonable middle ground for the linear SVM. We also include a second SVM variant that uses the RBF (radial basis function) kernel — a technique we will explain fully in Section 3. For now, the key idea is that while the linear SVM draws straight boundaries, the RBF kernel lets the SVM draw curved boundaries that can wrap around clusters in feature space, much like KNN adapts locally. Think of it as giving the SVM the flexibility of KNN while keeping the margin principle.

SVM vs Prior Methods — LOSO Across Tasks and Feature Sets

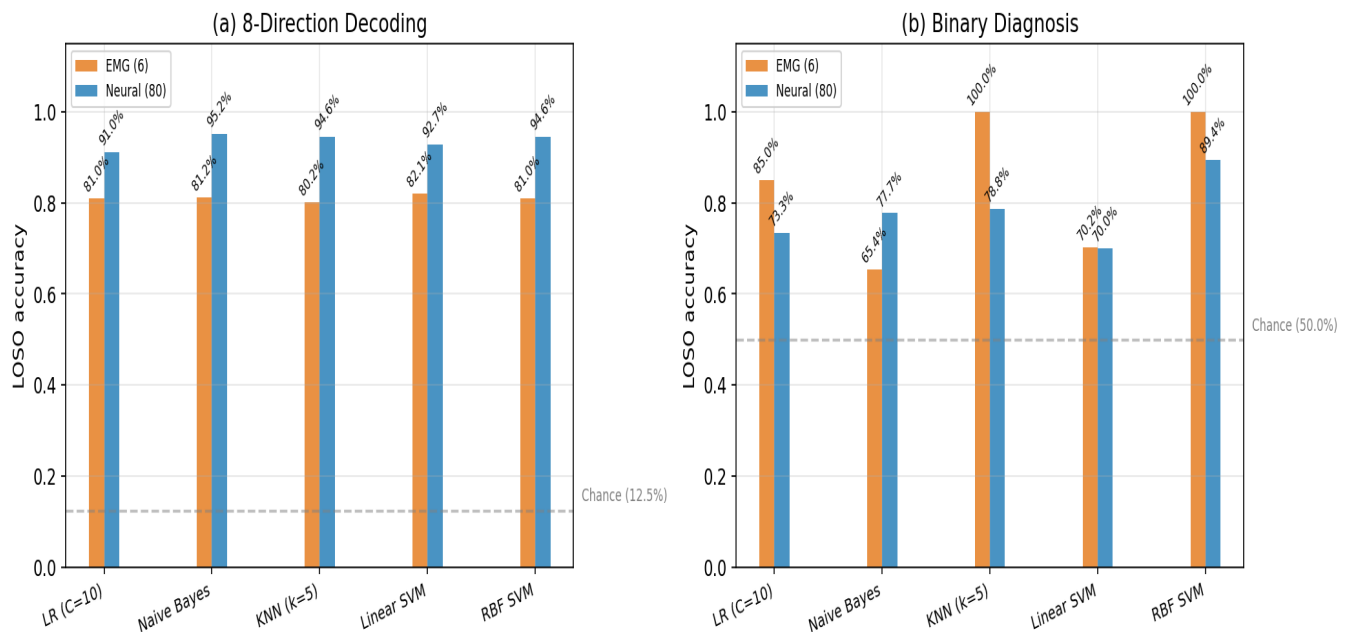


Figure 4. LOSO accuracy across all five methods and both feature sets (Linear SVM at  $C = 1$ ; RBF SVM at  $C = 10$ ,  $\gamma = \text{scale}$ ). Left: 8-direction decoding. Right: binary diagnosis (healthy vs impaired).

A note on  $C$ : the  $C$  sweep in Figure 3 selected  $C = 0.001$  as optimal for direction decoding, but that value is catastrophic for binary diagnosis — it gives 0% accuracy on both feature sets. Why? At  $C = 0.001$  the margin penalty dominates so completely that the SVM prefers an enormous margin with every trial allowed to violate it. For an 8-class problem with well-separated clusters, this still works because the broad structure is preserved. But for a binary task where healthy and impaired trials are interleaved in feature space, the slack overwhelms the signal and the classifier predicts a single class for every trial. This is a concrete lesson: a hyperparameter tuned on one

task can fail on another. We use  $C = 1$  in Figure 4 as a sensible middle ground.

Three results stand out. First, on 8-direction neural decoding, the linear SVM (92.7%) slightly improves over LR (91.0%), while the RBF SVM (94.6%) ties KNN and Naive Bayes at the top. The margin principle works as advertised.

Second, the linear SVM struggles on binary diagnosis — only 70% on both feature sets, the worst of all five methods. This is not a bug. The healthy-vs-impaired distinction in our data is driven by synergy merging: impaired subjects use fewer independent muscle coordination patterns, which shows up as nonlinear differences in the feature distributions. A straight hyperplane cannot capture this structure. Logistic regression does better (85% EMG) because its probabilistic framework is more forgiving of overlap, and KNN achieves 100% because it can wrap around each cluster locally.

Third, the RBF SVM rescues the binary task — jumping to 100% on EMG, matching KNN. This previews the power of kernels: by implicitly mapping the data to a higher-dimensional space where the classes become linearly separable, the RBF kernel recovers what the linear SVM misses. We explain how in Section 3.



### 3. The Kernel Trick

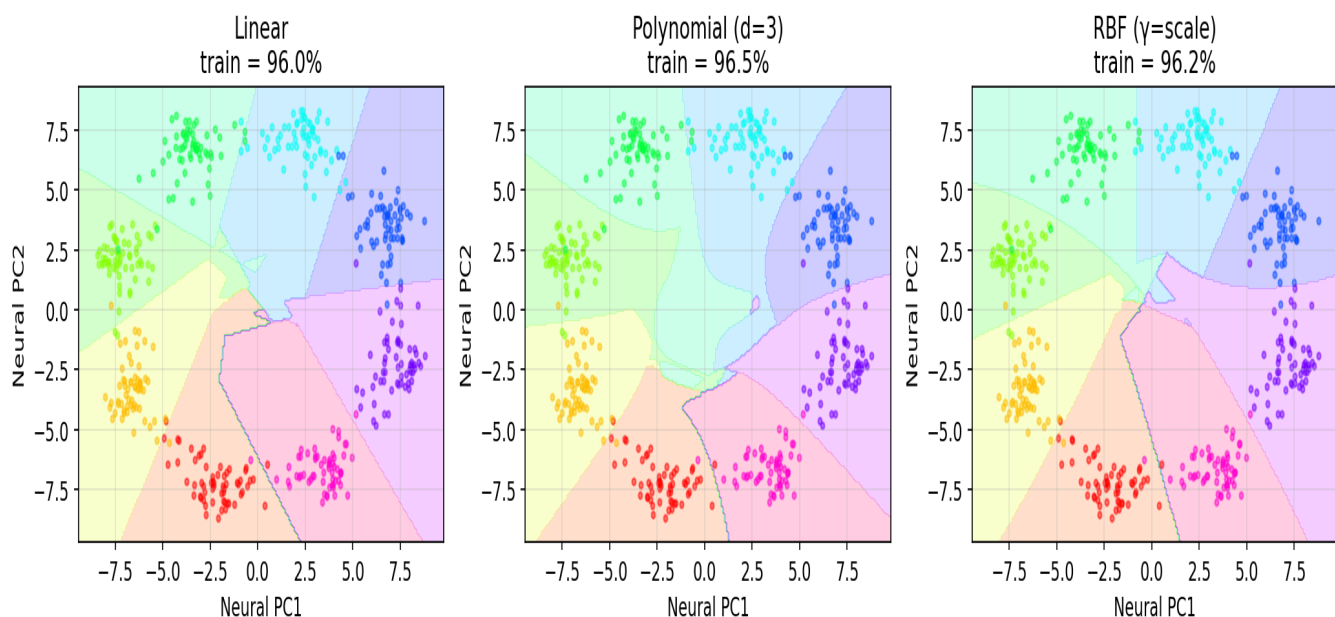
#### How can a linear method capture nonlinear boundaries?

Section 2 exposed a limitation: the linear SVM achieved only 70% on binary diagnosis because the healthy-vs-impaired distinction is nonlinear — it depends on synergy merging, not a simple shift in mean activation. Yet the RBF SVM jumped to 100%. How did it draw curved boundaries while still maximising a margin?

The answer is the kernel trick. Instead of fitting a linear boundary in the original feature space, we implicitly map the data to a higher-dimensional space where the classes become linearly separable — and then fit the maximum-margin hyperplane there. The key insight is that the SVM dual formulation (Section 1) depends on the data only through dot products  $x_i \cdot x_j$ . If we replace each dot product with a kernel function  $K(x_i, x_j)$  that computes the dot product in the mapped space, we never need to compute the mapped coordinates themselves.

Three kernels are commonly used — and the choice of kernel is itself a hyperparameter, just like  $C$  or  $k$  in KNN. It is a decision we make before training, and we select it via cross-validation. The Linear kernel,  $K(x, x') = x \cdot x'$ , is just the standard dot product — equivalent to the linear SVM we already tested. It works well for direction decoding because cosine tuning creates approximately linear class boundaries in neural PCA space. The Polynomial kernel,  $K(x, x') = (\gamma x \cdot x' + r)^d$ , captures interactions between features up to degree  $d$  — for example, co-contraction patterns where two muscles activate together during a particular reach direction. The RBF (Gaussian) kernel,  $K(x, x') = \exp(-\gamma \|x - x'\|^2)$ , measures similarity between neural population states: two trials with similar firing-rate patterns get a high kernel value regardless of which direction they encode. It maps to an infinite-dimensional space, making it the most flexible — and the most popular choice for neural data.

Kernel SVM: Decision Boundaries for Neural Direction Decoding



*Figure 5. Decision boundaries for three kernel SVMs on neural direction decoding in PCA space. The RBF kernel captures curved boundaries between adjacent directions that the linear kernel cannot.*

## Why kernels work: a conceptual illustration

Consider a classic toy example: two concentric rings of data. In the original 2D space, no straight line can separate them. But the RBF kernel implicitly maps these points to a higher-dimensional space where a hyperplane can cleanly divide the classes.

### The Kernel Trick: Implicit Mapping to Higher Dimensions

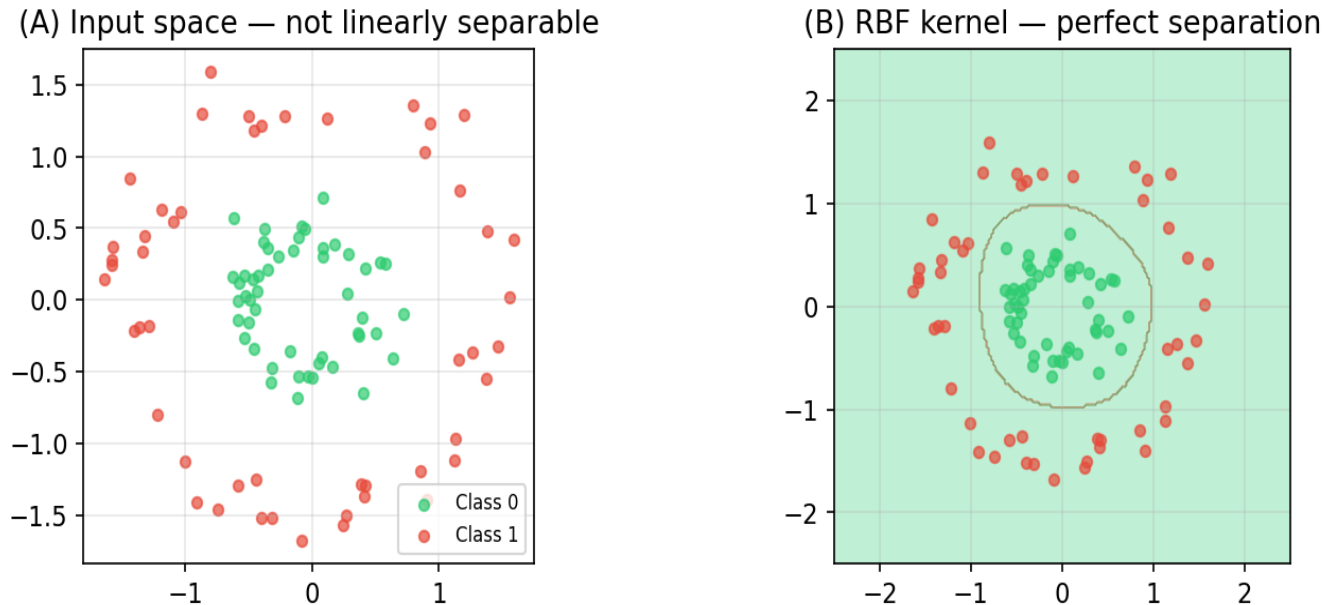


Figure 6. (A) Two classes arranged in concentric rings — no linear boundary exists. (B) An RBF-kernel SVM finds a circular decision boundary by implicitly operating in a higher-dimensional feature space.

The beauty of the kernel trick is computational: we never compute the (possibly infinite-dimensional) mapped features  $\phi(x)$ . The SVM optimisation depends only on dot products  $\phi(x_i) \cdot \phi(x_j)$ , which the kernel function gives us directly. Training cost scales with the number of samples, not the dimensionality of the implicit feature space.

Our binary diagnosis task is not as clean as concentric rings, but the principle is the same. Healthy and impaired subjects overlap in EMG space in a way that no single hyperplane can separate — the impairment shows up as synergy merging, which compresses the feature distribution nonlinearly. The RBF kernel maps these overlapping clusters to a space where a hyperplane can separate them, which is why RBF SVM jumped from 70% (linear) to 100% on EMG binary diagnosis in Figure 4.

## Choosing a kernel: practical guidelines

Start with linear. When the number of features is large relative to the number of trials, a linear kernel often suffices and avoids overfitting. In our dataset we have 80 neurons and 480 trials — not a desperately high-dimensional regime, but the linear kernel still works well for direction decoding because cosine tuning creates approximately linear class boundaries in neural space.

Try RBF when linear falls short. If linear performance plateaus or the task involves nonlinear structure (like our binary diagnosis), the RBF kernel is the default next step. It has one additional

hyperparameter,  $\gamma$ , that controls how far each support vector's influence extends. Small  $\gamma$  means broad influence (smoother boundaries, fewer support vectors); large  $\gamma$  means narrow influence (more complex boundaries, higher risk of overfitting to individual trials).

Hyperparameter tuning for RBF SVMs typically involves a 2D grid search over  $C$  and  $\gamma$ , using LOO cross-validation. A common starting grid is  $C \in \{0.1, 1, 10, 100\}$  and  $\gamma \in \{0.001, 0.01, 0.1, 1\}$ . scikit-learn's default  $\gamma = 1/(n_{\text{features}} \times \text{variance})$ , called 'scale', is often a good starting point — it is what we used for the RBF SVM in Figure 4.

## 4. Dimensionality Revisited

### Why do SVMs survive high dimensions better than KNN?

In Week 8 we saw that KNN degrades catastrophically as irrelevant features are added — every noisy dimension dilutes the meaningful distance signal. How do SVMs fare? To test this, we augment our 6 real EMG features with increasing numbers of pure noise dimensions and measure LOSO accuracy for each method.

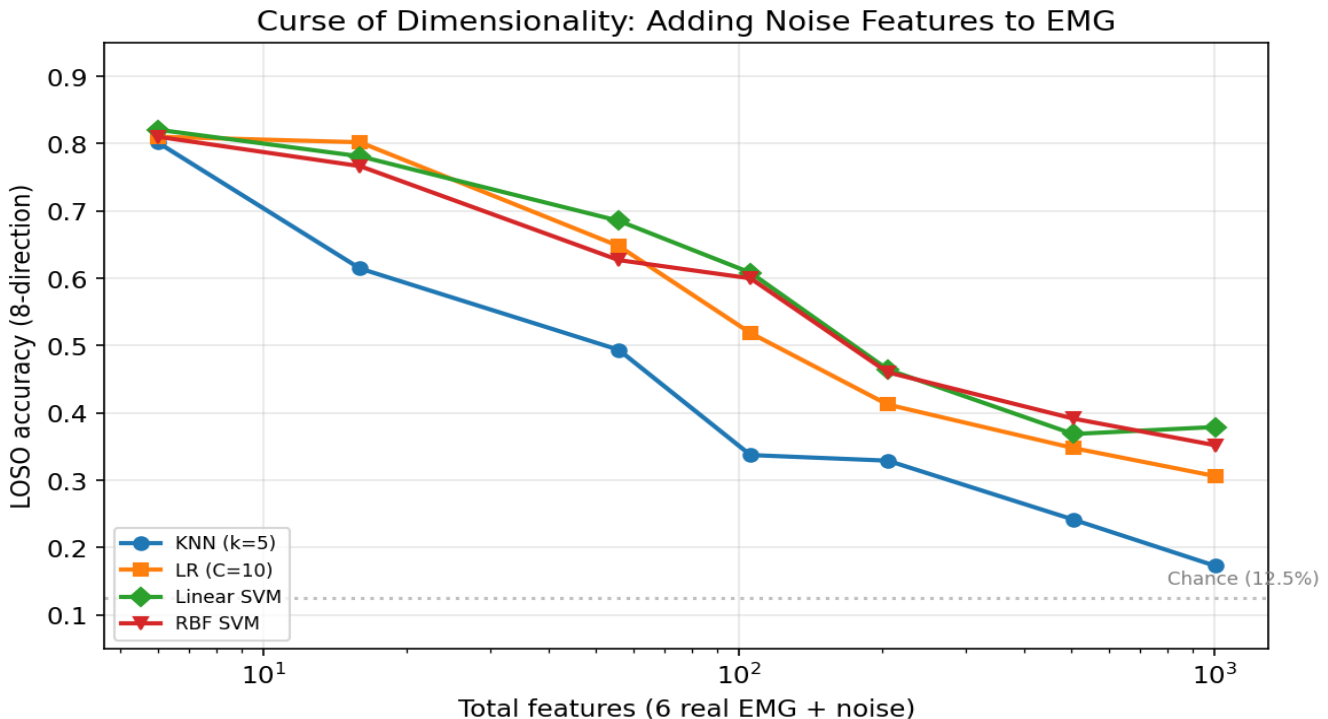


Figure 7. Accuracy vs total feature dimensions (6 real EMG + noise). KNN collapses first. Linear SVM and Logistic Regression are more resilient. The RBF SVM also degrades, but less steeply than KNN — the margin provides implicit regularisation.

Why do SVMs handle high dimensions relatively well? The SVM objective cares about the margin, not the raw dimensionality. The margin depends on the support vectors — a small subset of training points — so irrelevant features that do not affect these critical points have limited impact. The C parameter provides explicit regularisation against overfitting.

### The VC-dimension perspective

This result connects to a deeper question raised by Section 3: if the RBF kernel maps our data to an infinite-dimensional space, why doesn't the SVM overfit immediately? After all, with enough dimensions any training set can be memorised. The answer lies in a concept called the VC-dimension (Vapnik-Chervonenkis dimension), which measures the effective complexity of a classifier — how many arbitrary labelling patterns it can fit perfectly.

For a maximum-margin classifier, the VC-dimension is bounded by  $R^2/\rho^2$ , where  $R$  is the radius of the smallest ball enclosing all the data and  $\rho$  is the margin width. Crucially, this bound does not depend on the number of features — it depends only on how tightly the data is packed ( $R$ ) relative to how wide the margin is ( $\rho$ ). A wide margin means low effective complexity, even in

infinite dimensions.

Think about what this means for our reaching data. When we add 1000 noise features to the 6 real EMG channels, the data cloud's radius  $R$  grows (because each noise dimension adds random spread), but the margin  $\rho$  stays roughly the same because the noise features do not help separate the classes. The ratio  $R^2/\rho^2$  increases, so the VC-dimension goes up and performance degrades — but much more slowly than KNN, which treats every dimension as equally important for distance. KNN's effective complexity grows directly with dimensionality; the SVM's grows only as the margin shrinks.

This is why SVMs became popular for neural decoding tasks where the number of recording channels is large. With modern high-density arrays like Neuropixels (384 channels), dimensionality reduction before classification is still good practice — but SVMs are more forgiving than KNN if a few uninformative channels slip through.

## 5. Confusion Matrices

### Which reach directions are systematically confusable?

When a subject reaches to  $0^\circ$ , they activate a population of cosine-tuned neurons whose preferred directions cluster around  $0^\circ$ . But a  $45^\circ$  reach activates many of the same neurons — just at slightly lower rates — because cosine tuning curves are broad. The further apart two targets are, the less their population patterns overlap. This means we should expect classifiers to confuse adjacent directions ( $0^\circ$  vs  $45^\circ$ ) far more often than opposite directions ( $0^\circ$  vs  $180^\circ$ ). A confusion matrix lets us check this prediction directly.

How is the matrix computed? For each of the 20 LOSO folds, we train the classifier on 19 subjects and predict the held-out subject's 24 trials. We collect all 480 predictions across all folds, then build an  $8 \times 8$  table: row  $i$  is the true direction, column  $j$  is the predicted direction, and each cell contains the fraction of row  $i$ 's trials that were predicted as direction  $j$ . A perfect classifier has 1.00 on the diagonal and 0.00 everywhere else. Off-diagonal values reveal which directions the classifier confuses — and by extension, which directions share similar neural population patterns.

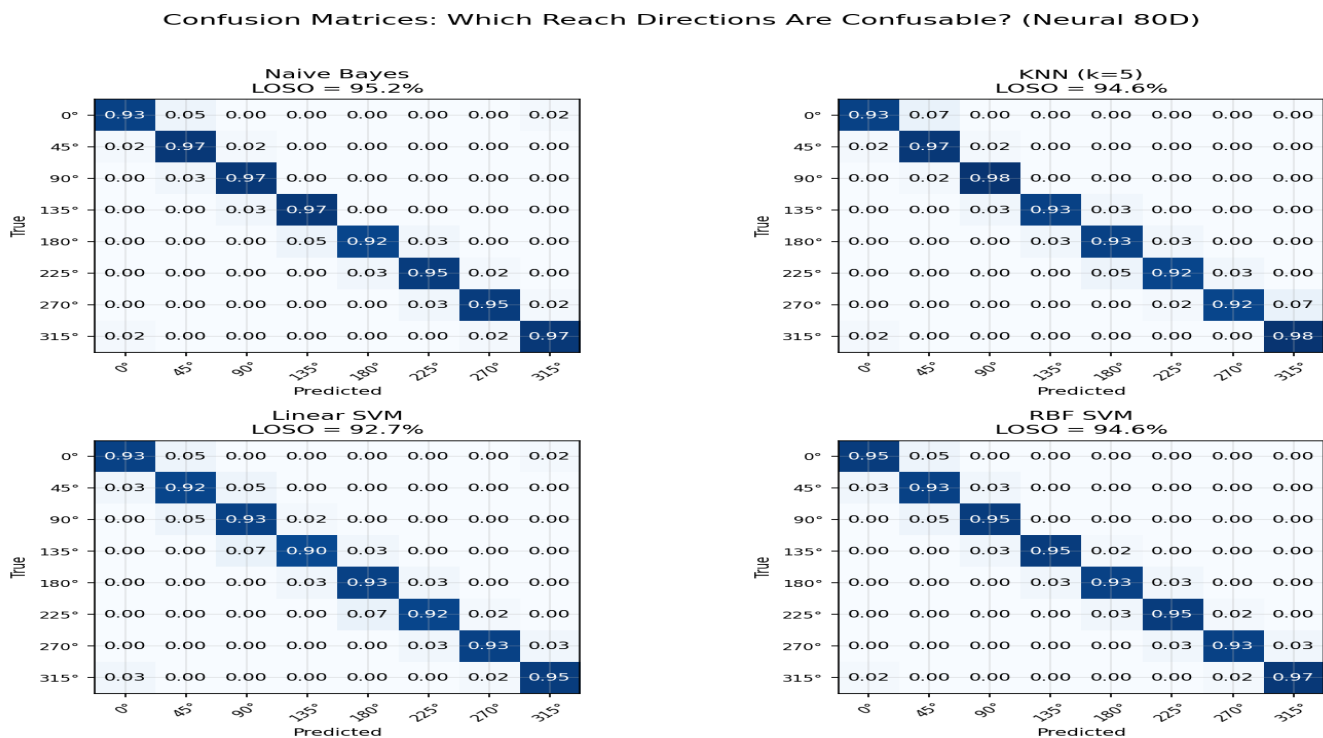


Figure 8. Normalised confusion matrices for four methods on neural 8-direction decoding (LOSO,  $C = 1$  for Linear SVM). Diagonal entries are correct classifications. Off-diagonal errors concentrate between adjacent directions (e.g.,  $0^\circ \leftrightarrow 45^\circ$ ,  $0^\circ \leftrightarrow 315^\circ$ ), consistent with the overlap of cosine-tuned population vectors for nearby targets.

The pattern is consistent across all four methods: errors are almost exclusively between neighbouring directions. This is not a classifier failure — it reflects the underlying neuroscience. Cosine tuning means that adjacent targets produce population activity patterns that genuinely overlap. No classifier can perfectly separate patterns that the motor cortex itself does not fully distinguish. The confusion matrix makes this biomechanical constraint visible.

### Per-direction F1 scores

Overall accuracy treats all errors equally, but in a reaching task some directions may be systematically harder to decode. The F1 score captures this: for each direction, it asks two questions. Precision: when the classifier says “90°,” how often is it actually 90°? Recall: of all the actual 90° trials, how many did the classifier catch? F1 combines both into a single number (their harmonic mean). A direction with high F1 is both reliably detected and rarely confused with other directions.

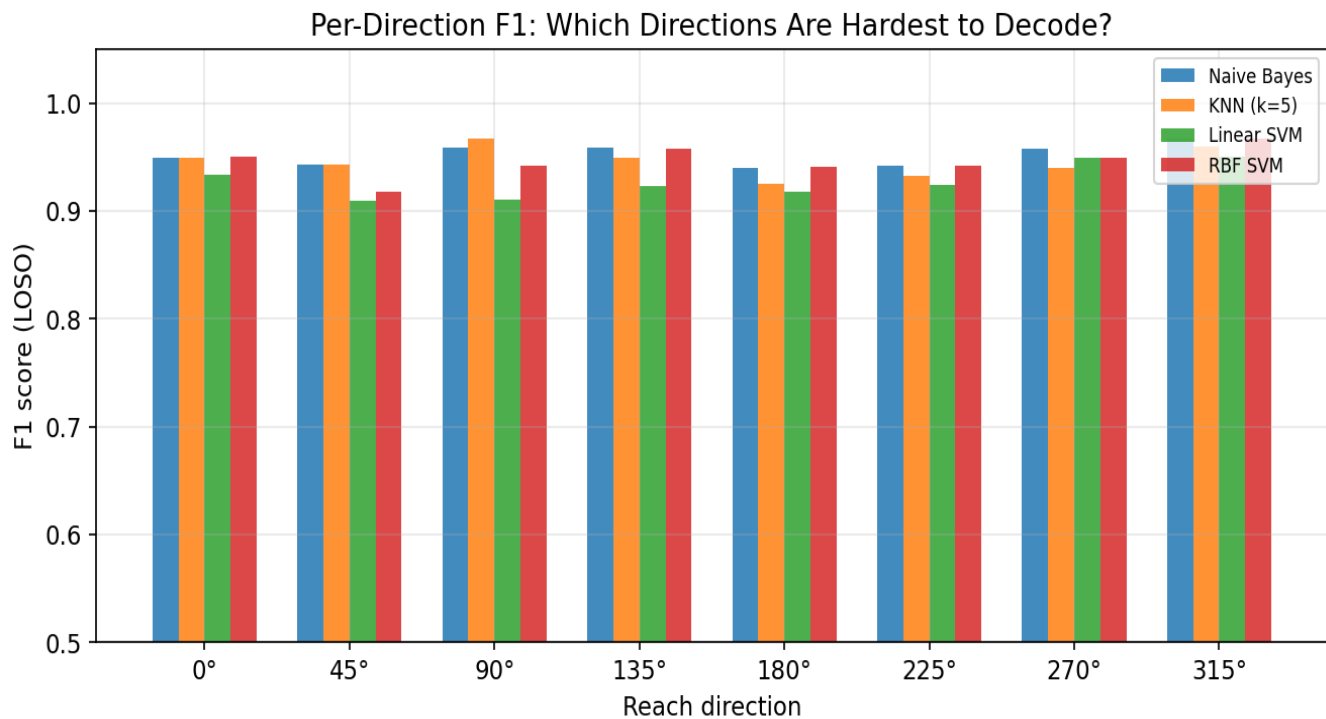


Figure 9. Per-direction F1 scores across four methods. All directions decode above 0.90, but the linear SVM shows slightly more variation — 45° and 180° dip compared to cardinal directions (0°, 90°). The RBF SVM achieves the most uniform F1 across all eight directions.

In a BCI for cursor or robotic arm control, directional errors are not equally costly. Decoding 0° as 45° moves the effector in roughly the right direction; decoding it as 180° moves it backwards. A clinician designing a BCI might use these per-direction F1 scores to decide whether 8 directions are too ambitious for a given patient — perhaps collapsing to 4 cardinal directions would eliminate the confusable pairs and give more reliable control.



## 6. From Offline Decoding to the BCI Pipeline

### When does margin translate to real-time robustness?

Everything we have done so far is offline analysis: we recorded data, took it back to the lab, and evaluated classifiers on pre-recorded trials. A real BCI must decode movement intention in real time, producing a prediction every few tens of milliseconds while the subject is actively reaching. The critical practical challenge is that the signal you calibrated on drifts over the course of a session.

Consider a typical motor cortex recording session. A Utah array is implanted in M1, and on day one you run a calibration block — the subject makes center-out reaches while you record 80 neurons. You train your SVM. By hour two of the same session, electrode impedances have shifted, a few neurons have drifted or dropped out, and the subject's attention and cortical excitability have changed. The feature vector for a 90° reach is no longer exactly where it was during calibration. If your classifier's boundary passes close to that trial, even a small drift flips the prediction. This is the scenario the SVM margin was designed for — a wide margin buys time before the drift crosses the boundary.

### Testing robustness to drift

We simulate this electrode drift by adding Gaussian noise of increasing magnitude to the test features without retraining the classifier. This mimics what happens in a real session: the calibration model stays fixed, but the incoming signals shift.

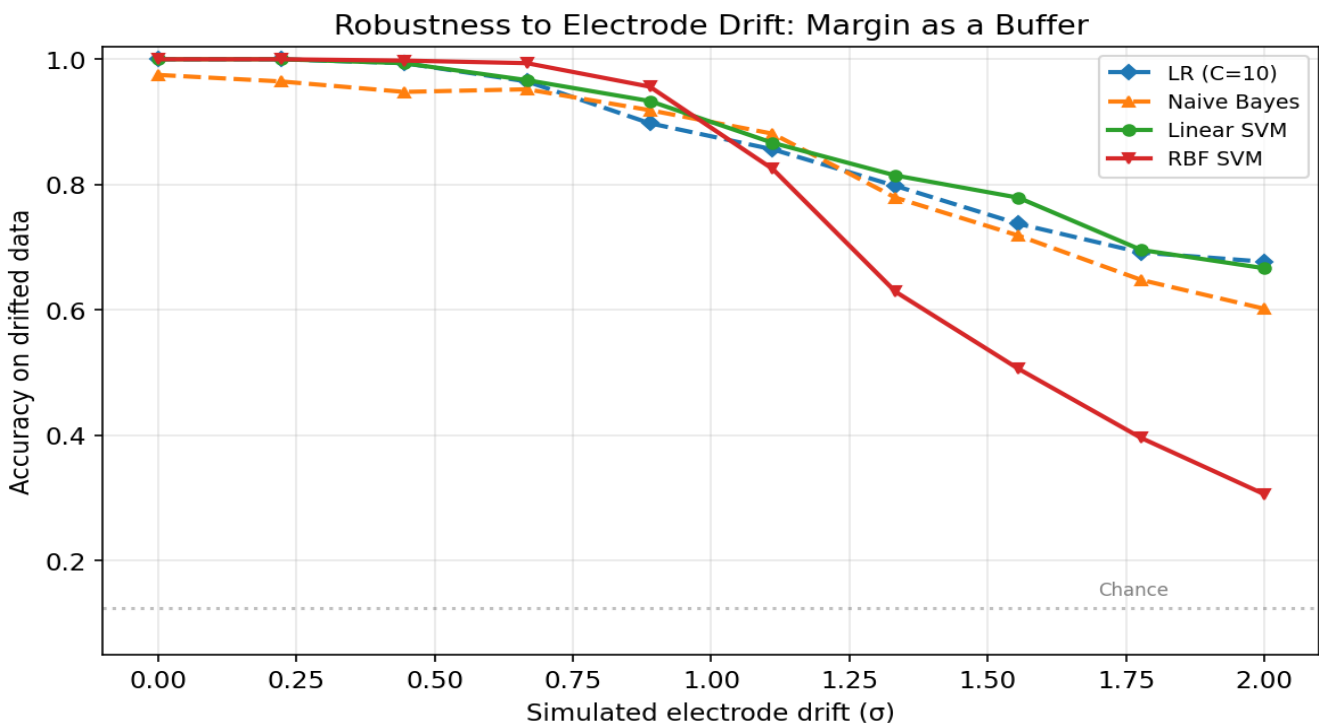


Figure 10. Accuracy under simulated electrode drift. At zero drift, all methods perform near ceiling. As drift increases, the Linear SVM degrades most gracefully among the linear methods. The RBF SVM, despite high accuracy at zero drift, drops faster — its complex curved boundaries are more sensitive to shifts in feature space.

This is an important and perhaps counterintuitive result. The RBF SVM was our top performer in Section 2, but it is the least robust to drift. Its curved boundaries fit the training data tightly, so when the data shifts, the predictions break. The Linear SVM's simpler, straight boundary with a wide margin tolerates more perturbation. In a clinical BCI that must work for hours without recalibration, this trade-off between peak accuracy and robustness is a real design decision.

## Computational cost

A BCI decoder must also be fast enough. Users perceive delays above ~200 ms as sluggish, and most systems aim for prediction latencies under 50 ms. How do our methods compare?

Computational Cost: Training vs Prediction for Real-Time BCI

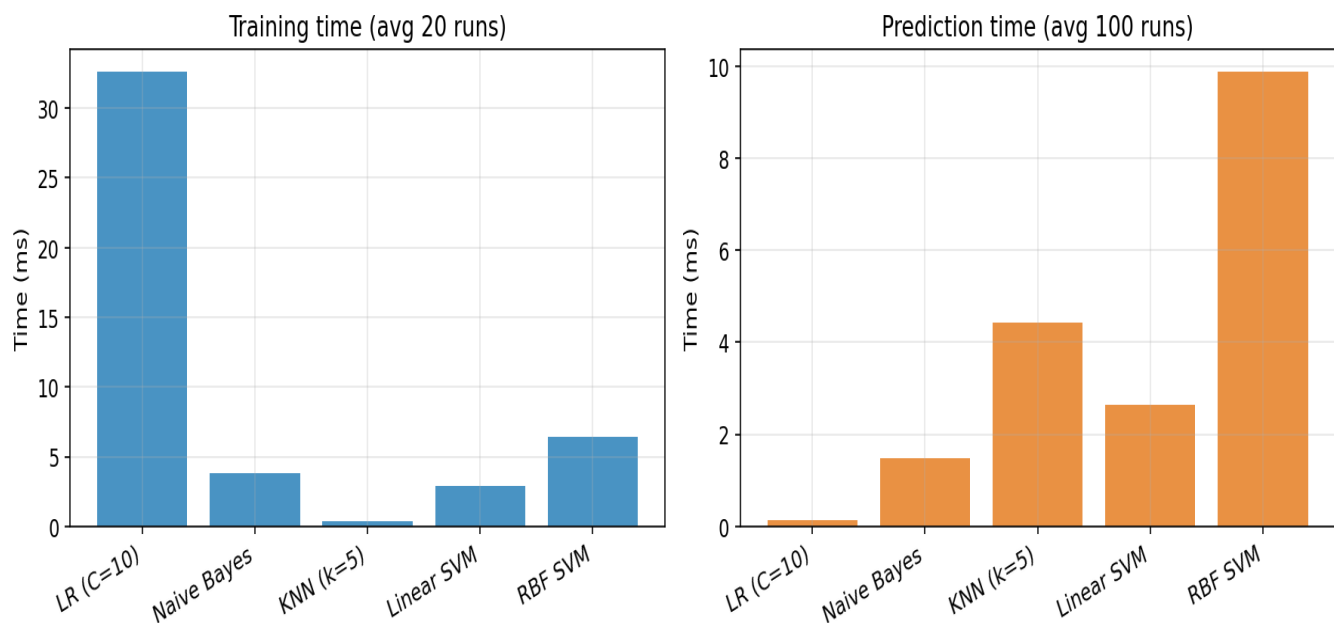


Figure 11. Training and prediction time for each method. Logistic Regression and Naive Bayes predict in under 2 ms. KNN is slower at prediction because it computes distances to all training points. The Linear SVM predicts in ~3 ms; the RBF SVM takes ~10 ms because prediction scales with the number of support vectors. All methods are fast enough for real-time BCI at these data scales.

## Prediction confidence and the decision function

Unlike Logistic Regression, a standard SVM does not output probabilities. Instead, the decision function  $f(x) = w \cdot x + b$  gives a signed distance to the hyperplane. Points far from the boundary have large  $|f(x)|$ ; points near the boundary have small  $|f(x)|$ .

This distance is directly useful in a BCI. A common strategy is confidence-gated decoding: only execute a movement command if  $|f(x)|$  exceeds a threshold. When the neural signal is ambiguous — the trial sits near the margin boundary — the system withholds the command rather than risk sending the cursor in the wrong direction. This reduces erroneous commands at the cost of slightly slower throughput. For safety-critical applications like wheelchair control, the trade-off is worth it: a missed command is far less dangerous than a wrong one.

scikit-learn offers Platt scaling (`SVC(probability=True)`) to convert SVM decision values into calibrated probabilities via a sigmoid fit, if downstream components require probability outputs.

## 7. Bringing It Together

We now have five classifiers in our toolbox, spanning linear and non-linear boundaries, parametric and non-parametric models, probabilistic and margin-based objectives. Each makes different trade-offs along axes that matter for neural decoding: accuracy, speed, robustness, interpretability, and data efficiency.

The SVM's contribution is the margin principle — the idea that among all correct separators, the one with the largest gap to the nearest data points generalises best. This principle is supported by statistical learning theory (Vapnik-Chervonenkis theory), which bounds the generalisation error in terms of the margin width rather than the number of parameters.

### Running Comparison Table Through Week 9

| Method      | EMG 8-Dir | Neural 8-Dir | EMG Binary | Neural Binary |
|-------------|-----------|--------------|------------|---------------|
| LR (C=10)   | 81.0%     | 91.0%        | 85.0%      | 73.3%         |
| Naive Bayes | 81.2%     | 95.2%        | 65.4%      | 77.7%         |
| KNN (k=5)   | 80.2%     | 94.6%        | 100.0%     | 78.8%         |
| Linear SVM  | 82.1%     | 92.7%        | 70.2%      | 70.0%         |
| RBF SVM     | 81.0%     | 94.6%        | 100.0%     | 89.4%         |

*Figure 12. Summary comparison of all classifiers studied so far. SVM rows are highlighted. The RBF SVM achieves the best overall accuracy on this dataset.*

### When to use what?

**Naive Bayes:** The fastest method to train and predict — under 4 ms for both on our 480-trial dataset. Best when features are approximately independent; PCA decorrelation rescues accuracy when muscle synergies violate this assumption (Week 7). Good first baseline for any new dataset.

**Logistic Regression:** Training is slower (~33 ms) due to iterative optimisation, but prediction is the fastest of all methods (~0.2 ms per trial). Use when you need calibrated probabilities (e.g., for confidence-gated BCI output) or when L1 regularisation is needed for feature selection.

**k-NN:** No training phase at all — it simply stores the data. But prediction is expensive (~4.5 ms) because it computes distances to every training trial. Best in low dimensions with abundant data. Degrades catastrophically in high dimensions (Week 8).

**Linear SVM:** Moderate training cost (~3 ms), fast prediction (~3 ms). The margin provides the best robustness to electrode drift among all methods tested (Figure 10). Scales well to high

dimensions. Use as the default when you want a reliable, robust linear classifier.

RBF SVM: Highest peak accuracy on our data, but the slowest prediction (~10 ms) because it must evaluate the kernel against every support vector. More hyperparameters ( $C$ ,  $\gamma$ ) require careful tuning via grid search. Less robust to drift than the linear SVM (Figure 10). Use when nonlinear boundaries are needed and you can afford the tuning.

## What we learned this week

- SVMs find the maximum-margin boundary — the separator with the largest gap to the nearest training trials. The margin provides a quantifiable robustness buffer against electrode drift, signal non-stationarity, and inter-session variability.
- Only a subset of training trials (support vectors) determine the boundary. For our 8-direction task, 206 of 480 trials (43%) are support vectors; the rest could be deleted without changing the classifier.
- Hyperparameters ( $C$ , kernel,  $\gamma$ ) are choices made before training, unlike parameters ( $w$ ,  $b$ ) learned from data. A hyperparameter tuned on one task can fail on another —  $C = 0.001$  was optimal for direction decoding but catastrophic (0%) for binary diagnosis.
- The kernel trick maps data to a higher-dimensional space where linear separation becomes possible, without ever computing the mapped coordinates. The RBF kernel rescued binary diagnosis from 70% (linear) to 100%.
- The VC-dimension bound ( $R^2/\rho^2$ ) explains why SVMs survive high dimensions: the margin width, not the number of features, controls effective model complexity.
- The linear SVM is the most robust method to electrode drift (Figure 10), while the RBF SVM achieves the highest peak accuracy but degrades fastest under drift. This trade-off is a real design decision for clinical BCIs.

## Looking ahead

We have now covered the major “classical” classifiers. Next week (Week 10) we move to ensemble methods — Random Forests and boosting — which combine many weak learners into something stronger. The intuitions developed here — margins, kernels, and the bias-variance trade-off — will reappear in new forms.

Modern BCI research increasingly uses deep learning, but SVMs remain the workhorse for scenarios with limited training data (< 1000 trials), fast recalibration requirements, and interpretability constraints — all common in clinical settings.

*Next week — Week 10: Random Forests & Ensemble Methods. Can we combine many weak learners into something stronger?*